In [ ]:
```
import json
import marimo as mo
import numpy as np
import plotly.graph_objects as go
import polars as pl
import xgboost as xgb
from pathlib import Path
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import precision_recall_curve, roc_curve
from sklearn.model_selection import StratifiedKFold, cross_val_predict
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
```

1. Data and model config loading

In [ ]:
```
PROJECT_ROOT = Path(__file__).resolve().parents[1]
DATA_PATH = PROJECT_ROOT / "data" / "processed" / "loans_processed.parquet"
RAW_DATA_PATH = PROJECT_ROOT / "data" / "raw" / "mlcasestudy Final.csv"
PARAMS_PATH = PROJECT_ROOT / "data" / "models" / "best_xgb_params.json"

df = pl.read_parquet(DATA_PATH)

raw_df = pl.read_csv(RAW_DATA_PATH, try_parse_dates=True)
outstanding_21d = (
    df.select("loan_id")
    .join(raw_df.select("loan_id", "amount_outstanding_21d"), on="loan_id")[
        "amount_outstanding_21d"
    ]
    .to_numpy()
)

TARGET = "default_21d"
NON_FEATURE_COLUMNS = ["loan_id", "loan_issue_date", "default_14d", TARGET]
CATEGORICAL_FEATURES = ["merchant_group", "merchant_category"]

feature_df = df.drop(NON_FEATURE_COLUMNS)
numeric_feature_df = feature_df.drop(CATEGORICAL_FEATURES)

y = df[TARGET].to_numpy().astype(int)
loan_amounts = df["loan_amount"].to_numpy()

xgb_params = json.loads(PARAMS_PATH.read_text())
```

2. Cross-validated predictions on the full dataset

To evaluate portfolio-level metrics without leakage, we use `cross_val_predict` to obtain out-of-fold probability estimates for every row. Each loan gets a prediction from a model that never saw it during training.

The XGBoost model uses the best hyperparameters found by Optuna in notebook 02.

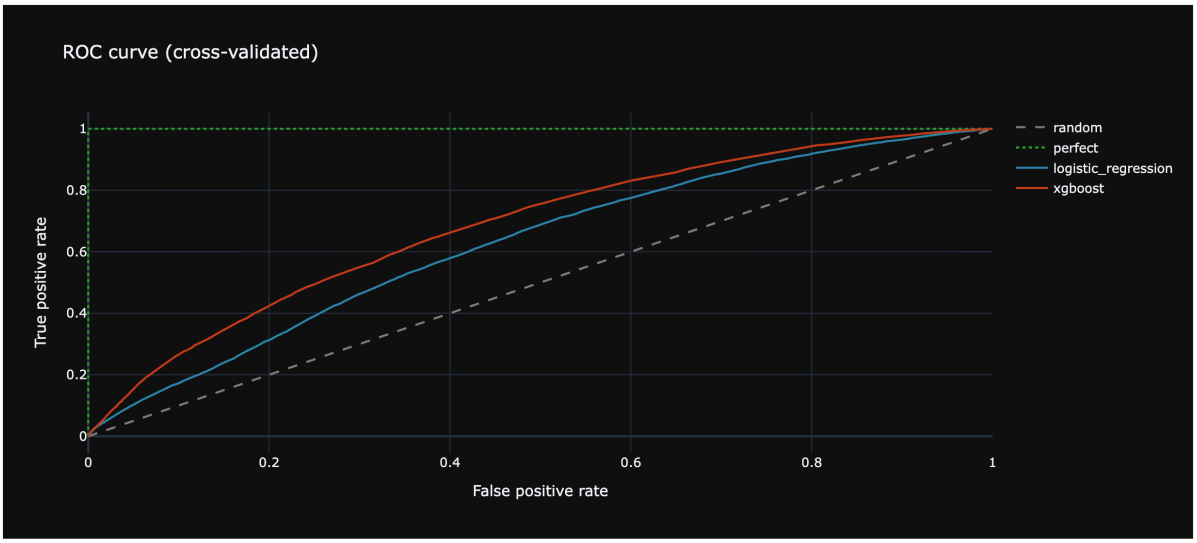
In [ ]:
```
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

lr_pipeline = Pipeline(
    [
        ("scaler", StandardScaler()),
        ("lr", LogisticRegression(max_iter=1000, random_state=42)),
    ]
)
X_numeric = numeric_feature_df.to_numpy().astype(np.float64)
lr_cv_probs = cross_val_predict(
    lr_pipeline, X_numeric, y, cv=cv, method="predict_proba"
)[:, 1]

X_full_pd = feature_df.to_pandas()
for _col in CATEGORICAL_FEATURES:
    X_full_pd[_col] = X_full_pd[_col].astype("category")

xgb_model = xgb.XGBClassifier(**xgb_params)
xgb_cv_probs = cross_val_predict(
    xgb_model, X_full_pd, y, cv=cv, method="predict_proba"
)[:, 1]

perfect_probs = y.astype(np.float64)
```

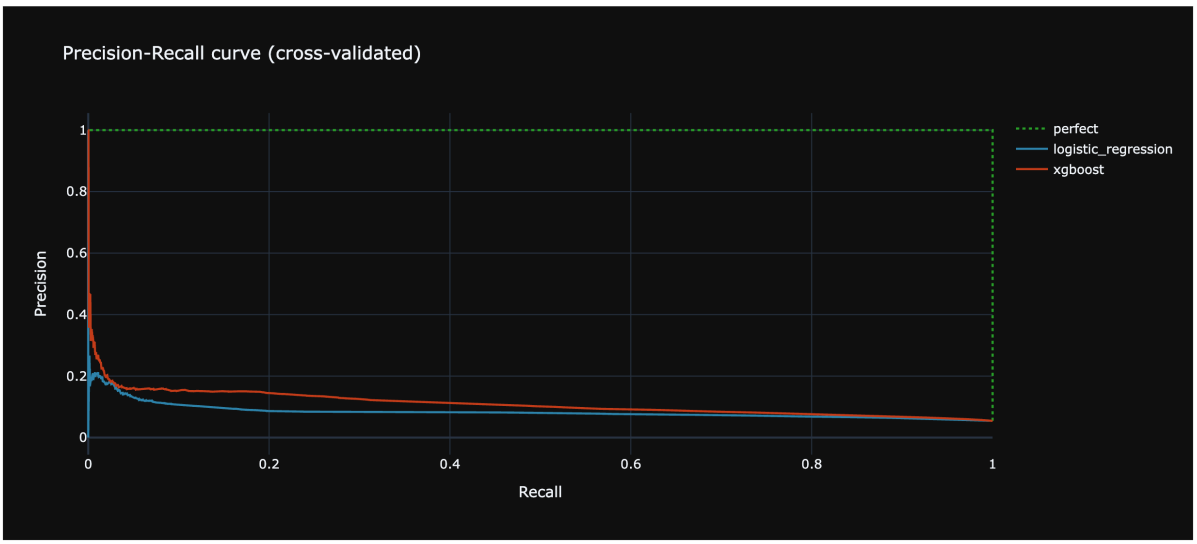
3. ROC and Precision-Recall curves

In [ ]:
```
_lr_fpr, _lr_tpr, _ = roc_curve(y, lr_cv_probs)
_xgb_fpr, _xgb_tpr, _ = roc_curve(y, xgb_cv_probs)
_pf_fpr, _pf_tpr, _ = roc_curve(y, perfect_probs)

roc_fig = go.Figure()
roc_fig.add_trace(
    go.Scatter(
        x=[0, 1],
        y=[0, 1],
        mode="lines",
        line={"dash": "dash", "color": "gray"},
        name="random",
    )
)
roc_fig.add_trace(
    go.Scatter(
        x=_pf_fpr,
        y=_pf_tpr,
        mode="lines",
        name="perfect",
        line={"dash": "dot", "color": "#2CA02C"},
```

```
    )
)
roc_fig.add_trace(
    go.Scatter(
        x=_lr_fpr,
        y=_lr_tpr,
        mode="lines",
        name="logistic_regression",
        line={"color": "#2E86AB"},
    )
)
roc_fig.add_trace(
    go.Scatter(
        x=_xgb_fpr,
        y=_xgb_tpr,
        mode="lines",
        name="xgboost",
        line={"color": "#C73E1D"},
    )
)
roc_fig.update_layout(
    title="ROC curve (cross-validated)",
    xaxis_title="False positive rate",
    yaxis_title="True positive rate",
    height=500,
)

_lr_prec, _lr_rec, _ = precision_recall_curve(y, lr_cv_probs)
_xgb_prec, _xgb_rec, _ = precision_recall_curve(y, xgb_cv_probs)
_pf_prec, _pf_rec, _ = precision_recall_curve(y, perfect_probs)

pr_fig = go.Figure()
pr_fig.add_trace(
    go.Scatter(
        x=_pf_rec,
        y=_pf_prec,
        mode="lines",
        name="perfect",
        line={"dash": "dot", "color": "#2CA02C"},
    )
)
pr_fig.add_trace(
    go.Scatter(
        x=_lr_rec,
        y=_lr_prec,
        mode="lines",
        name="logistic_regression",
        line={"color": "#2E86AB"},
    )
)
pr_fig.add_trace(
    go.Scatter(
        x=_xgb_rec,
        y=_xgb_prec,
        mode="lines",
        name="xgboost",
```

```
        line={"color": "#C73E1D"},
    )
)
pr_fig.update_layout(
    title="Precision-Recall curve (cross-validated)",
    xaxis_title="Recall",
    yaxis_title="Precision",
    height=500,
)

mo.vstack([mo.ui.plotly(roc_fig), mo.ui.plotly(pr_fig)])
```





4. Threshold sweep

For each decision threshold, a loan is **approved** if the predicted default probability is below the threshold, and **rejected** otherwise. We sweep thresholds to compare how each model trades off approval rate against portfolio quality.

In [ ]:
```
def threshold_sweep(y_true, y_prob, amounts, thresholds):
    rows = []
```

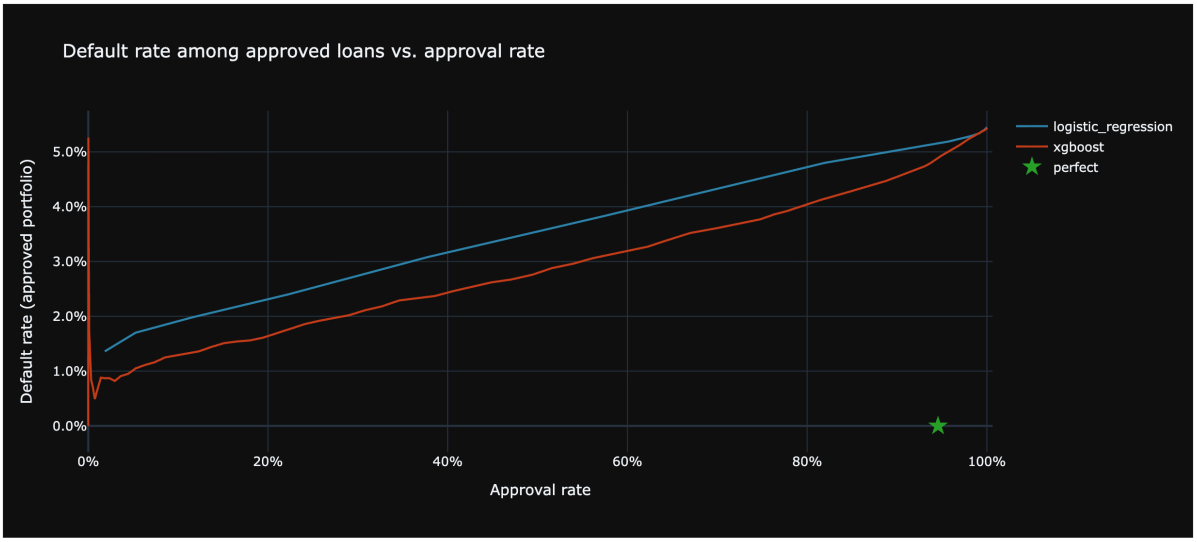
```
    n = len(y_true)
    total_defaults = y_true.sum()
    for t in thresholds:
        approved = y_prob < t
        n_approved = approved.sum()
        if n_approved == 0:
            continue
        approval_rate = n_approved / n
        defaults_in_approved = y_true[approved].sum()
        default_rate_approved = defaults_in_approved / n_approved
        defaults_caught = y_true[~approved].sum()
        catch_rate = defaults_caught / total_defaults if total_defaults > 0
        approved_volume = amounts[approved].sum()
        defaulted_volume = amounts[approved & (y_true == 1)].sum()
        rows.append(
            {
                "threshold": round(t, 3),
                "approval_rate": round(approval_rate, 4),
                "default_rate_approved": round(default_rate_approved, 4),
                "catch_rate": round(catch_rate, 4),
                "n_approved": int(n_approved),
                "defaults_in_approved": int(defaults_in_approved),
                "approved_volume": int(approved_volume),
                "defaulted_volume": int(defaulted_volume),
            }
        )
    return pl.DataFrame(rows)

thresholds = np.arange(0.01, 1.0, 0.01)
```

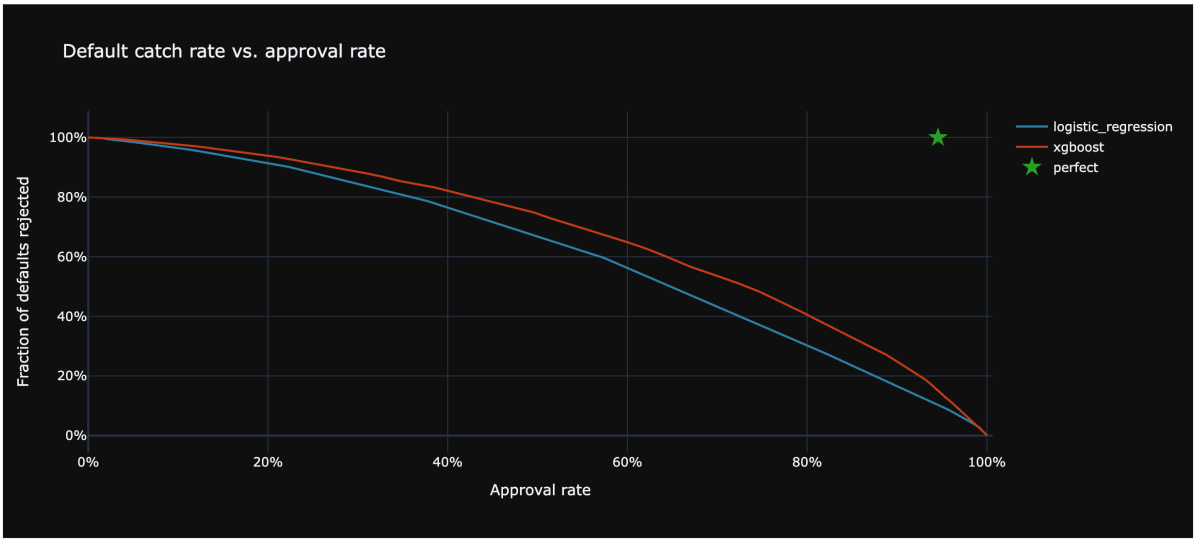
In [ ]:
```
lr_sweep = threshold_sweep(y, lr_cv_probs, loan_amounts, thresholds)
xgb_sweep = threshold_sweep(y, xgb_cv_probs, loan_amounts, thresholds)
perfect_sweep = threshold_sweep(y, perfect_probs, loan_amounts, thresholds)
```

In [ ]:
```
_pf_approval = float(perfect_sweep["approval_rate"][0])
_pf_default = float(perfect_sweep["default_rate_approved"][0])
_pf_catch = float(perfect_sweep["catch_rate"][0])

_approval_fig = go.Figure()
_approval_fig.add_trace(
    go.Scatter(
        x=lr_sweep["approval_rate"].to_list(),
        y=lr_sweep["default_rate_approved"].to_list(),
        mode="lines",
        name="logistic_regression",
        line={"color": "#2E86AB"},
        customdata=lr_sweep["threshold"].to_list(),
        hovertemplate="threshold: %{customdata}<br>approval: %{x:.1%}<br>def
    )
)
_approval_fig.add_trace(
    go.Scatter(
        x=xgb_sweep["approval_rate"].to_list(),
        y=xgb_sweep["default_rate_approved"].to_list(),
        mode="lines",
```

```
        name="xgboost",
        line={"color": "#C73E1D"},
        customdata=xgb_sweep["threshold"].to_list(),
        hovertemplate="threshold: %{customdata}<br>approval: %{x:.1%}<br>def
    )
)
_approval_fig.add_trace(
    go.Scatter(
        x=[_pf_approval],
        y=[_pf_default],
        mode="markers",
        name="perfect",
        marker={"color": "#2CA02C", "size": 14, "symbol": "star"},
    )
)

_approval_fig.update_layout(
    title="Default rate among approved loans vs. approval rate",
    xaxis_title="Approval rate",
    yaxis_title="Default rate (approved portfolio)",
    xaxis_tickformat=".0%",
    yaxis_tickformat=".1%",
    height=500,
)

_catch_fig = go.Figure()
_catch_fig.add_trace(
    go.Scatter(
        x=lr_sweep["approval_rate"].to_list(),
        y=lr_sweep["catch_rate"].to_list(),
        mode="lines",
        name="logistic_regression",
        line={"color": "#2E86AB"},
    )
)
_catch_fig.add_trace(
    go.Scatter(
        x=xgb_sweep["approval_rate"].to_list(),
        y=xgb_sweep["catch_rate"].to_list(),
        mode="lines",
        name="xgboost",
        line={"color": "#C73E1D"},
    )
)
_catch_fig.add_trace(
    go.Scatter(
        x=[_pf_approval],
        y=[_pf_catch],
        mode="markers",
        name="perfect",
        marker={"color": "#2CA02C", "size": 14, "symbol": "star"},
    )
)

_catch_fig.update_layout(
    title="Default catch rate vs. approval rate",
```

```
    xaxis_title="Approval rate",
    yaxis_title="Fraction of defaults rejected",
    xaxis_tickformat=".0%",
    yaxis_tickformat=".0%",
    height=500,
)

mo.vstack([mo.ui.plotly(_approval_fig), mo.ui.plotly(_catch_fig)])
```





5. Financial impact

The profit model uses actual data where possible:

- **Revenue**: every approved loan (defaulted or not) earns a merchant fee, modeled as a percentage of the loan amount. Klarna charges merchants for "buy now pay later" — the consumer pays no interest.
- **Loss**: each approved default loses the actual outstanding balance at 21 days (`amount_outstanding_21d`), minus whatever fraction is eventually recovered through collections.

Both rates are adjustable below.

In [ ]:
```
merchant_fee_slider = mo.ui.slider(
    start=0.01,
    stop=0.10,
    step=0.005,
    value=0.03,
    label="Merchant fee rate",
)
recovery_rate_slider = mo.ui.slider(
    start=0.0,
    stop=1.0,
    step=0.05,
    value=0.0,
    label="Recovery rate (fraction of outstanding recovered)",
)
```

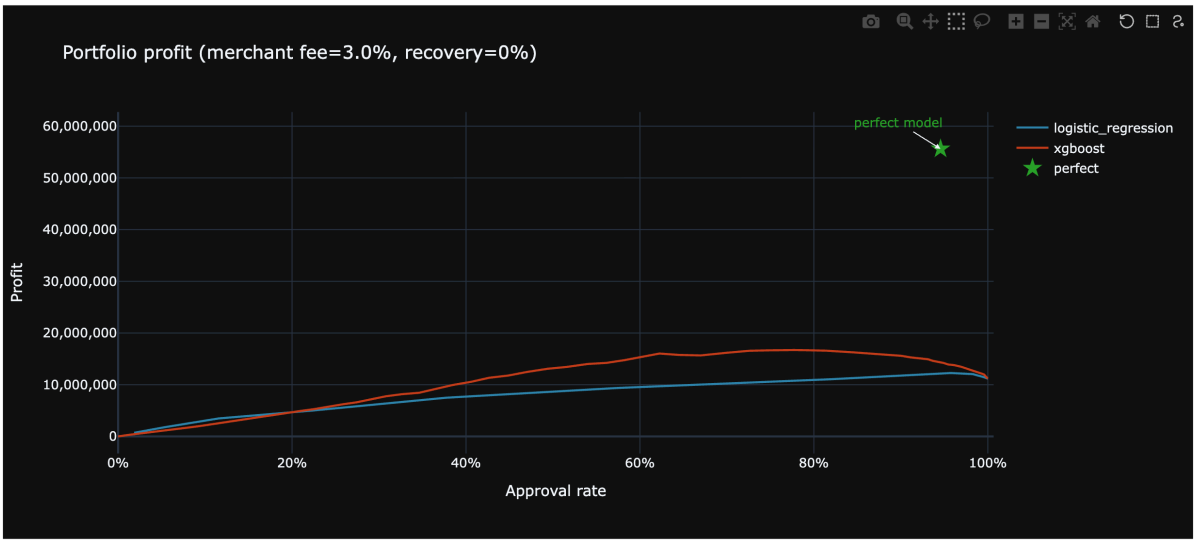
In [ ]:
```
_merchant_fee = merchant_fee_slider.value
_recovery_rate = recovery_rate_slider.value

def _compute_profit(
    y_true, y_prob, amounts, outstanding, t, merchant_fee, recovery_rate
):
    approved = y_prob < t
    fee_revenue = (amounts[approved] * merchant_fee).sum()
    credit_loss = (
        outstanding[approved & (y_true == 1)] * (1 - recovery_rate)
    ).sum()
    return float(fee_revenue - credit_loss)

_profit_rows = []
for _t in thresholds:
    _lr_p = _compute_profit(
        y,
        lr_cv_probs,
        loan_amounts,
        outstanding_21d,
        _t,
        _merchant_fee,
        _recovery_rate,
    )
    _xgb_p = _compute_profit(
        y,
        xgb_cv_probs,
        loan_amounts,
        outstanding_21d,
        _t,
        _merchant_fee,
        _recovery_rate,
    )
    _pf_p = _compute_profit(
        y,
        perfect_probs,
        loan_amounts,
        outstanding_21d,
```

```
        _t,
        _merchant_fee,
        _recovery_rate,
    )
    _n_lr = int((lr_cv_probs < _t).sum())
    _n_xgb = int((xgb_cv_probs < _t).sum())
    _n_pf = int((perfect_probs < _t).sum())
    _profit_rows.append(
        {
            "threshold": round(_t, 3),
            "lr_profit": round(_lr_p),
            "xgb_profit": round(_xgb_p),
            "perfect_profit": round(_pf_p),
            "lr_approval_rate": round(_n_lr / len(y), 4),
            "xgb_approval_rate": round(_n_xgb / len(y), 4),
            "perfect_approval_rate": round(_n_pf / len(y), 4),
        }
    )

profit_df = pl.DataFrame(_profit_rows)

_lr_best = profit_df.sort("lr_profit", descending=True).head(1)
_xgb_best = profit_df.sort("xgb_profit", descending=True).head(1)
_pf_best = profit_df.sort("perfect_profit", descending=True).head(1)

_pf_profit = float(_pf_best["perfect_profit"][0])
_pf_ar = float(_pf_best["perfect_approval_rate"][0])

_profit_fig = go.Figure()
_profit_fig.add_trace(
    go.Scatter(
        x=profit_df["lr_approval_rate"].to_list(),
        y=profit_df["lr_profit"].to_list(),
        mode="lines",
        name="logistic_regression",
        line={"color": "#2E86AB"},
        customdata=profit_df["threshold"].to_list(),
        hovertemplate="threshold: %{customdata}<br>approval: %{x:.1%}<br>pro
    )
)
_profit_fig.add_trace(
    go.Scatter(
        x=profit_df["xgb_approval_rate"].to_list(),
        y=profit_df["xgb_profit"].to_list(),
        mode="lines",
        name="xgboost",
        line={"color": "#C73E1D"},
        customdata=profit_df["threshold"].to_list(),
        hovertemplate="threshold: %{customdata}<br>approval: %{x:.1%}<br>pro
    )
)
_profit_fig.add_trace(
    go.Scatter(
        x=[_pf_ar],
        y=[_pf_profit],
        mode="markers",
```

```
        name="perfect",
        marker={"color": "#2CA02C", "size": 14, "symbol": "star"},
    )
)
_profit_fig.add_annotation(
    x=_pf_ar,
    y=_pf_profit,
    text="perfect model",
    showarrow=True,
    arrowhead=2,
    ax=-40,
    ay=-25,
    font={"color": "#2CA02C", "size": 12},
)
_profit_fig.update_layout(
    title=f"Portfolio profit (merchant fee={_merchant_fee:.1%}, recovery={_r
    xaxis_title="Approval rate",
    yaxis_title="Profit",
    xaxis_tickformat=".0%",
    yaxis_tickformat=",",
    height=500,
)

optimal_thresholds = pl.concat(
    [
        _pf_best.select(
            pl.lit("perfect").alias("model"),
            pl.col("threshold").alias("best_threshold"),
            pl.col("perfect_approval_rate").alias("approval_rate"),
            pl.col("perfect_profit").alias("max_profit"),
        ),
        _lr_best.select(
            pl.lit("logistic_regression").alias("model"),
            pl.col("threshold").alias("best_threshold"),
            pl.col("lr_approval_rate").alias("approval_rate"),
            pl.col("lr_profit").alias("max_profit"),
        ),
        _xgb_best.select(
            pl.lit("xgboost").alias("model"),
            pl.col("threshold").alias("best_threshold"),
            pl.col("xgb_approval_rate").alias("approval_rate"),
            pl.col("xgb_profit").alias("max_profit"),
        ),
    ]
)
mo.vstack(
    [
        merchant_fee_slider,
        recovery_rate_slider,
        mo.ui.plotly(_profit_fig),
        optimal_thresholds,
    ]
)
```

Merchant fee rate

Recovery rate (fraction of outstanding recovered)



Search...

model str	best_threshold f64	approval_rate f64	max_profit i64
perfect	0.01	0.9456	55,617,563
logistic_regression	0.08	0.9575	12,264,166
xgboost	0.57	0.7634	16,726,675

3 rows, 4 columns

No selection

In [ ]:

In [ ]: